DBMS ASSIGNMENT

Database Design for an Event Ticketing System

Real-World Application: Event Ticketing and Venue Seating Platform

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	07 September 2026

1. Name and Purpose of the Application

Application Name: Event Ticketing and Venue Seating System

Purpose: This kind of application is what sits behind a concert, sports match, or theatre show when you go online to pick a seat and pay for it. A customer looks through upcoming events, picks a venue's seat map, books one or more specific seats, and pays for them. Behind that simple flow, the database has to keep track of who booked what, which seats belong to which venue, which of those seats are already taken for a given show, and whether the payment actually went through. The main problem it solves is making sure two people can't end up with the same seat for the same event.

2. Required Database Tables

Seven tables are enough to cover the whole booking process, from a customer creating an account to a seat being paid for. Anything beyond this (like refund policies or venue staff logins) isn't part of the core problem, so it's left out:

- 1. Users
- 2. Venues
- 3. Seats
- 4. Events
- 5. Bookings
- 6. Booking_Seats
- 7. Payments

3. Columns / Attributes for Each Table

Table 1: Users

Column	Data Type	Key	Purpose
user_id	INT	PK	Unique ID for each customer
name	VARCHAR		Customer name
phone	VARCHAR	UNIQUE	Customer phone number

Column	Data Type	Key	Purpose
email	VARCHAR	UNIQUE	Customer email
password_hash	VARCHAR		Stored login credential hash
created_at	DATETIME		Account creation date and time

Table 2: Venues

Column	Data Type	Key	Purpose
venue_id	INT	PK	Unique ID for each venue
name	VARCHAR		Venue name
address	VARCHAR		Venue address
city	VARCHAR		City where the venue is located
total_capacity	INT		Total number of seats in the venue

Table 3: Seats

Column	Data Type	Key	Purpose
seat_id	INT	PK	Unique ID for each seat
venue_id	INT	FK	Venue the seat belongs to
section	VARCHAR		Seating section, e.g. Left Block, Balcony
row_label	VARCHAR		Row identifier, e.g. A, B, C
seat_number	VARCHAR		Seat number within the row
seat_type	VARCHAR		Regular, Premium, or VIP

Table 4: Events

Column	Data Type	Key	Purpose
event_id	INT	PK	Unique ID for each event
venue_id	INT	FK	Venue where the event is held
title	VARCHAR		Event name
category	VARCHAR		Concert, sports, theatre, conference, etc.
event_date	DATE		Date the event takes place
start_time	TIME		Event start time
description	VARCHAR		Short description of the event

Table 5: Bookings

Column	Data Type	Key	Purpose
booking_id	INT	PK	Unique ID for each booking
user_id	INT	FK	Customer who made the booking
event_id	INT	FK	Event the booking is for
booking_time	DATETIME		Date and time the booking was made
total_amount	DECIMAL		Total amount for all seats in the booking
booking_status	VARCHAR		Pending, confirmed, or cancelled

Table 6: Booking_Seats

Column	Data Type	Key	Purpose
booking_seat_id	INT	PK	Unique ID for each booked seat
booking_id	INT	FK	Booking this seat belongs to
seat_id	INT	FK	Seat that was booked
event_id	INT	FK	Event for which the seat was booked
price	DECIMAL		Price paid for this specific seat
ticket_code	VARCHAR	UNIQUE	Unique code printed or scanned on the ticket

Table 7: Payments

Column	Data Type	Key	Purpose
payment_id	INT	PK	Unique payment ID
booking_id	INT	FK	Booking for which payment was made
amount	DECIMAL		Amount paid
payment_method	VARCHAR		UPI, card, net banking, etc.
payment_status	VARCHAR		Pending, successful, failed, refunded
paid_at	DATETIME		Payment date and time

4. Relationships Between the Tables

Relationship Summary

Tables	Relationship	Explanation
Venues → Seats	One-to-Many	Every seat belongs to one venue, but a venue obviously has lots of seats.
Venues → Events	One-to-Many	A venue hosts many events over time, though each event only happens at one venue.

Tables	Relationship	Explanation
Users → Bookings	One-to-Many	One user can make several bookings (for different events, or the same event more than once), but a booking belongs to just one user.
Events → Bookings	One-to-Many	An event will usually have many bookings against it; each booking, though, is for a single event.
Bookings → Booking_Seats	One-to-Many	If someone books three seats in one go, that's one booking but three Booking_Seats rows.
Seats → Booking_Seats	One-to-Many	The same physical seat gets reused across many different events, so it shows up in many booking-seat records over time.
Events → Booking_Seats	One-to-Many	Each booking-seat row also records which event it was for — that's what stops the same seat being booked twice for one show.
Bookings → Payments	One-to-Many	Normally a booking has just one payment, but keeping this as one-to-many leaves room for things like a failed attempt followed by a retry.

Simplified ER Relationship

Venues 1 ──< Seats

Venues 1 ──< Events

Users 1 ──< Bookings >── 1 Events

Bookings 1 ──< Booking_Seats >── 1 Seats

Booking_Seats >── 1 Events

Bookings 1 ──< Payments

5. Brief Purpose of Each Table

Table	Purpose
Users	Holds the customer's account details, basically what they'd need to log in and have a booking tied to their name.
Venues	Keeps a record of each venue — where it is and roughly how many people it can hold.
Seats	This is the venue's actual seat map: every seat it has, laid out by section and row, regardless of which event is happening there.
Events	One row per show or match — what it's called, what kind of event it is, and when and where it's happening.
Bookings	The main transaction table. Every time someone books, this is where it's logged: who booked, for which event, and how much it came to.

Table	Purpose
Booking_Seats	This is the table doing the real work — it ties a booking to the exact seats picked for a specific event, so a seat can't accidentally get sold to two different people for the same show.
Payments	Tracks whether a booking actually got paid for, and how.

6. Conclusion

An event ticketing and seating system like this can be built around seven tables. Bookings and Booking_Seats end up doing most of the heavy lifting, since between them they connect the user, the event, and the exact seats to one another, while Payments simply tracks whether the money came through. Primary keys keep each record unique, foreign keys tie related tables together, and the Booking_Seats table is really the piece that stops the same seat from being sold to two different people for the same show. Overall, this structure is enough to run seat-level ticket sales, event scheduling, and payments without needing anything more complicated.